

회의록을 정리하는 AI 서기

회의록을 정리하는 AI 서기

1. 음성을 텍스트로 변환하기
2. 로컬에서 음성을 텍스트로 변환하기
3. 문장과 화자 구분하기
4. 회의록을 정리하는 AI 서기 완성하기

3. 문장과 화자 구분하기

- 위스퍼 API는 음성을 잘 받아쓰지만 누가 어떤 말을 했는지 구분해 주지는 않음
- 이번 절에서는 여러 사람의 대화에서 화자를 구분하고 누가 어떤 말을 했는지 분석
- 화자를 구분하는 기능은 pyannote.audio를 이용해 개발
 - pyannote.audio는 화자를 분리하는 오픈소스 툴킷으로 파이토치 머신러닝 프레임워크에서 동작
 - <https://pypi.org/project/pyannote.audio/> 참조

3. 문장과 화자 구분하기

- (실습) 화자 분리 모델로 시간대별 화자 구분하기
 - 허깅페이스에서 화자 분리 모델 내려 받고 사용 준비하기
 - pyannote.audio 설치 (이미 설치되어 있음. 버전만 확인)

```
(.venv)
```

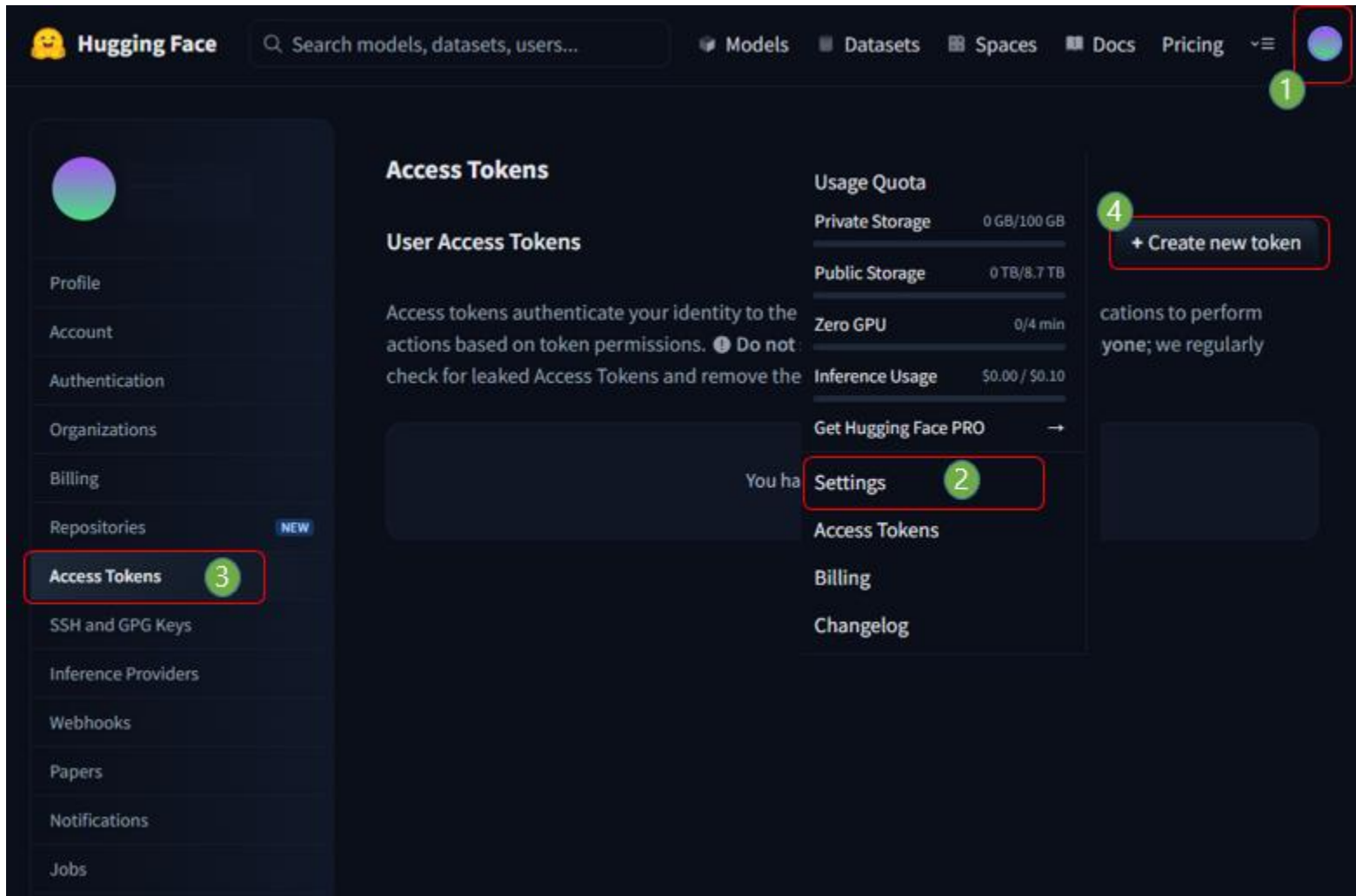
```
$ pip install pyannote.audio
```

3. 문장과 화자 구분하기

- (실습) 화자 분리 모델로 시간대별 화자 구분하기
 - 허깅페이스에서 화자 분리 모델 내려 받고 사용 준비하기

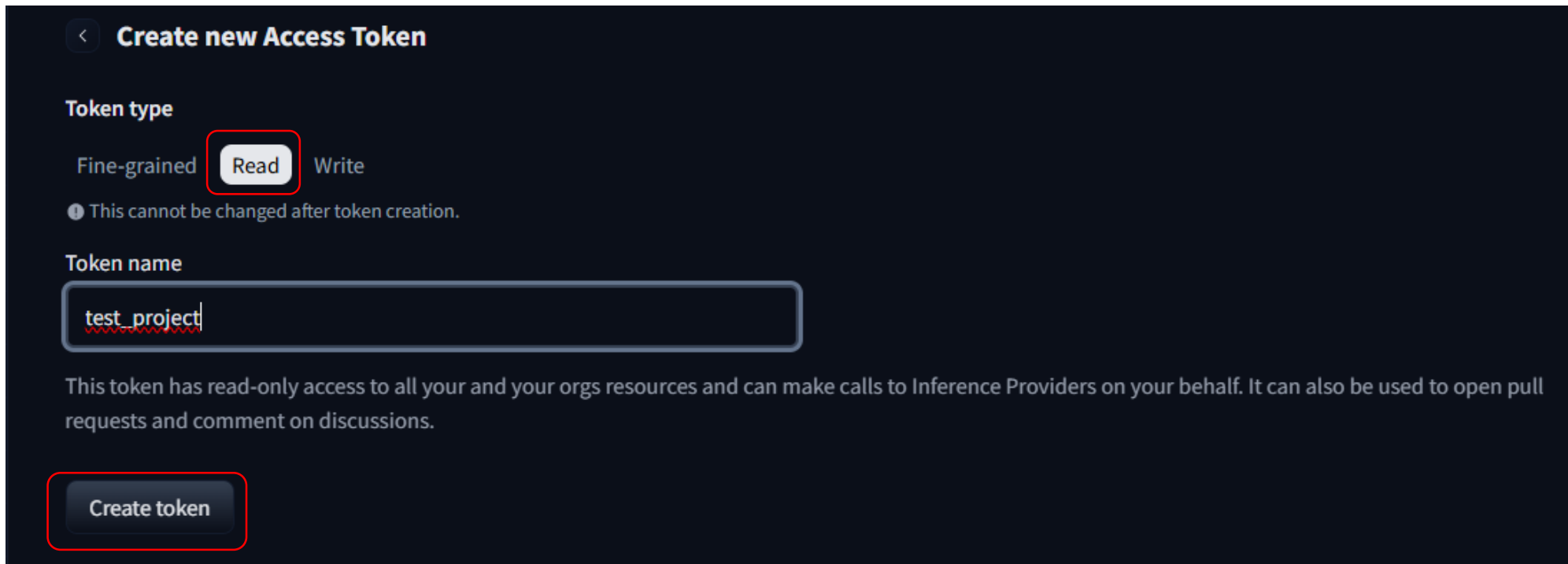
1. 허깅페이스 웹 사이트

(<https://huggingface.co>)에 로그인
→ 프로필 로고 클릭 → [Settings] 클릭
→ [Access Tokens] 선택
→ [+Create new token] 선택



3. 문장과 화자 구분하기

- (실습) 화자 분리 모델로 시간대별 화자 구분하기
 - 허깅페이스에서 화자 분리 모델 내려 받고 사용 준비하기
 1. 허깅페이스 웹 사이트



< Create new Access Token

Token type

Fine-grained **Read** Write

ⓘ This cannot be changed after token creation.

Token name

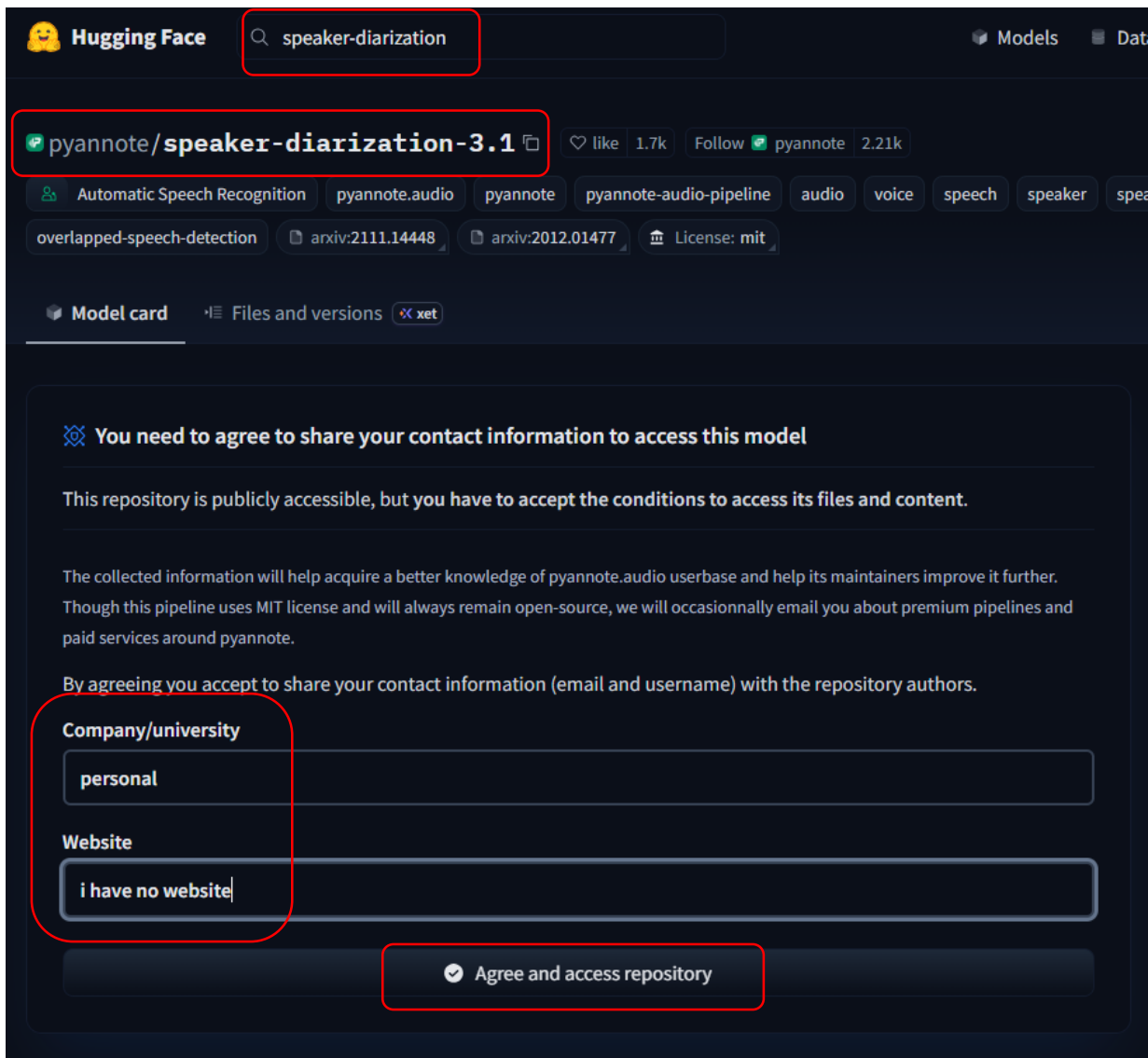
test_project

This token has read-only access to all your and your orgs resources and can make calls to Inference Providers on your behalf. It can also be used to open pull requests and comment on discussions.

Create token

3. 문장과 화자 구분하기

- (실습) 화자 분리 모델로 시간대별 화자 구분하기
 - 허깅페이스에서 화자 분리 모델 내려 받고 사용 준비하기
2. 허깅페이스 웹 사이트에서 'speaker-diarization' 검색
→ pyannote의 speaker-diarization-3.1 모델 찾기



3. 문장과 화자 구분하기

- (실습) 화자 분리 모델로 시간대별 화자 구분하기
 - 허깅페이스에서 화자 분리 모델 내려 받고 사용 준비하기

3. speaker_diarization.ipynb 파일 만들기

4. 모델 상세 페이지인 Usage 영역의 예제 코드를 참고해서 코드 작성 - **실행하면 오류 발생**

- pyannote/speaker-diarization-3.1 모델을 사용하려면 pyannote/segmentation-3.0의 사용 권한을 획득해야 함

```
# instantiate the pipeline
from pyannote.audio import Pipeline

pipeline = Pipeline.from_pretrained(
    "pyannote/speaker-diarization-3.1",
    use_auth_token="HUGGINGFACE_ACCESS_TOKEN_GOES_HERE"
)
```

오류 메시지 발생

```
c:\workspaces\ai_agent\src\gpt_agent\.venv\Lib\site-packages\tqdm\auto.py:21: TqdmWarning: IPProgress not found. Ple
from .autonotebook import tqdm as notebook_tqdm
```

```
Could not download 'pyannote/speaker-diarization-3.1' pipeline.
It might be because the pipeline is private or gated so make
sure to authenticate. Visit https://hf.co/settings/tokens to
create your access token and retry with:
```

```
>>> Pipeline.from_pretrained('pyannote/speaker-diarization-3.1',
...                           use_auth_token=YOUR_AUTH_TOKEN)
```

```
If this still does not work, it might be because the pipeline is gated:
visit https://hf.co/pyannote/speaker-diarization-3.1 to accept the user conditions.
```

3. 문장과 화자 구분하기

- (실습) 화자 분리 모델로 시간대별 화자 구분하기

- 허깅페이스에서 화자 분리 모델 내려 받고 사용 준비하기

5. 허깅페이스 웹 사이트의 `pyannote/speaker-diarization-3.1` 상세 페이지의 Requirements 부분에서 `[pyannote/segmentation-3.0]` 링크 클릭

Requirements

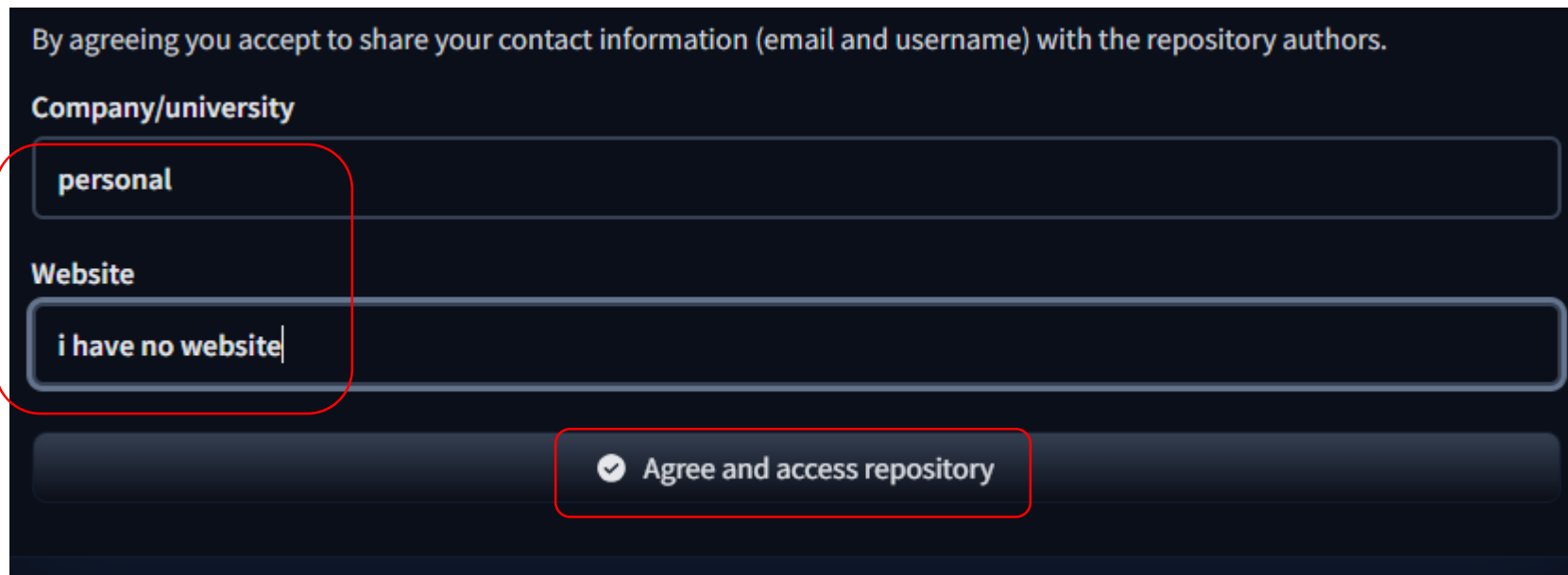
1. Install `pyannote.audio 3.1` with `pip install pyannote.audio`
2. Accept `pyannote/segmentation-3.0` user conditions
3. Accept `pyannote/speaker-diarization-3.1` user conditions
4. Create access token at hf.co/settings/tokens.

3. 문장과 화자 구분하기

- (실습) 화자 분리 모델로 시간대별 화자 구분하기

- 허깅페이스에서 화자 분리 모델 내려 받고 사용 준비하기

6. 소속 기관과 웹 사이트를 적고 [Agree and access repository] 클릭



By agreeing you accept to share your contact information (email and username) with the repository authors.

Company/university

personal

Website

i have no website

✓ Agree and access repository

The image shows a dark-themed web form. At the top, a line of text states: 'By agreeing you accept to share your contact information (email and username) with the repository authors.' Below this, there are two input fields. The first is labeled 'Company/university' and contains the text 'personal'. The second is labeled 'Website' and contains the text 'i have no website'. A red rounded rectangle highlights both of these input fields. At the bottom of the form, there is a button with a checkmark icon and the text 'Agree and access repository', which is also highlighted by a red rounded rectangle.

3. 문장과 화자 구분하기

- (실습) 화자 분리 모델로 시간대별 화자 구분하기

- 음성 파일에서 화자 분리하기

- 전기 기타를 배울 때 저렴한 기타와 비싼 기타 중에서 어떤 것으로 시작하는 것이 좋은지 AI와 토론한 MP3 파일을 사용

```
# run the pipeline on an audio file
diarization = pipeline("../audio/싼기타_비싼기타.mp3")

# dump the diarization output to disk using RTTM format
with open("../audio/싼기타_비싼기타.rttm", "w", encoding='utf-8') as rttm:
    diarization.write_rttm(rttm)
```

- rttm(Rich Transcription Time Marked)
 - 소리(특히 음성)를 분석한 결과를 기록하는 텍스트 파일 형식
 - 누가(speaker), 언제(start), 얼마나(duration) 말했는지 시간 기준으로 적어놓은 파일

3. 문장과 화자 구분하기

● (실습) 화자 분리 모델로 시간대별 화자 구분하기

• 음성 파일에서 화자 분리하기

- Type = SPEAKER : 이 줄이 무엇을 의미하는지
- File ID = file1 : 어떤 오디오 파일인지
- Channel = 1 : 오디오 채널 (입력 통로)
- Start Time : 말 시작 시간 (초)
- Duration : 말한 길이
- Orthography = <NA> :
 - 원래는 “실제 말한 텍스트”
 - diarization은 텍스트를 모르기 때문에 <NA> (없음)
- Speaker Type = <NA>
 - male/female/child -> 하지만 대부분 사용 안 함 → <NA>
- Speaker Name = speaker_0
- Confidence = <NA> : AI가 얼마나 확실하는지(pyannote는 보통 안 넣음)
- Lookahead = <NA> : 미래 정보 / 추가 정보(거의 사용 안 함)

RTTM 기본 구조 (한 줄)

SPEAKER file1 1 0.00 2.35 <NA> <NA> speaker_0 <NA> <NA>

총 10개 필드(칸)로 구성됨

10개 필드 완전 해석

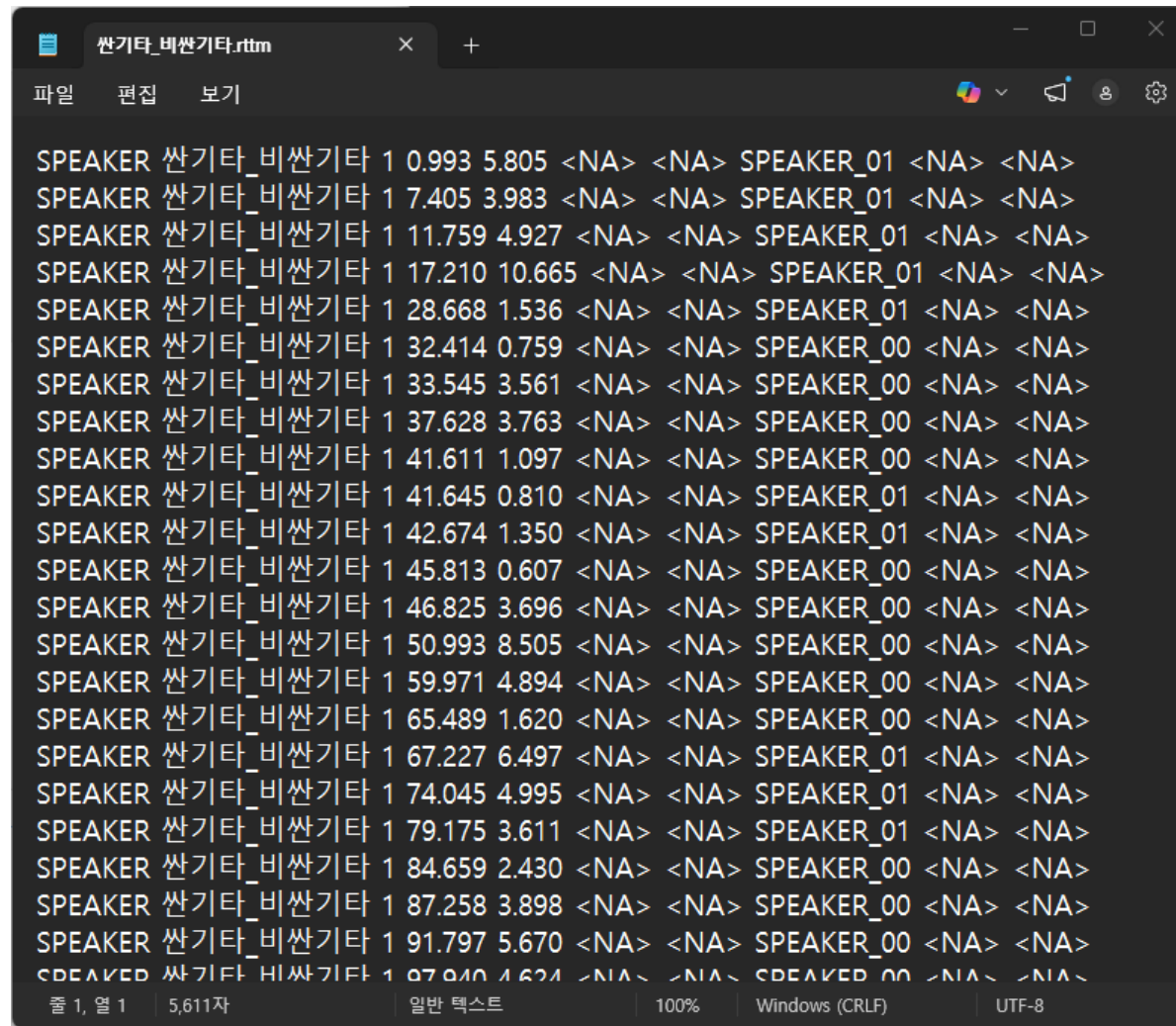
번호	예시 값	이름	의미
1	SPEAKER	Type	이 줄의 종류
2	file1	File ID	파일 이름
3	1	Channel	오디오 채널
4	0.00	Start Time	시작 시간 (초)
5	2.35	Duration	길이 (초)
6	<NA>	Orthography	실제 텍스트
7	<NA>	Speaker Type	화자 타입
8	speaker_0	Speaker Name	화자 ID
9	<NA>	Confidence	신뢰도
10	<NA>	Lookahead	추가 정보

3. 문장과 화자 구분하기

- (실습) 화자 분리 모델로 시간대별 화자 구분하기

- 음성 파일에서 화자 분리하기

- 화자 분리 실행 결과
- 의미 해석
 - SPEAKER_01의 발언은 0.993초부터 5.805초 동안,
그리고 7.405초부터 3.983초 동안 이어짐
 - 이런 식으로 SPEAKER_01의 발언이 계속되다가
32.414초부터 SPEAKER_00의 발언이 나옴



```
SPEAKER 싼기타_비싼기타 1 0.993 5.805 <NA> <NA> SPEAKER_01 <NA> <NA>
SPEAKER 싼기타_비싼기타 1 7.405 3.983 <NA> <NA> SPEAKER_01 <NA> <NA>
SPEAKER 싼기타_비싼기타 1 11.759 4.927 <NA> <NA> SPEAKER_01 <NA> <NA>
SPEAKER 싼기타_비싼기타 1 17.210 10.665 <NA> <NA> SPEAKER_01 <NA> <NA>
SPEAKER 싼기타_비싼기타 1 28.668 1.536 <NA> <NA> SPEAKER_01 <NA> <NA>
SPEAKER 싼기타_비싼기타 1 32.414 0.759 <NA> <NA> SPEAKER_00 <NA> <NA>
SPEAKER 싼기타_비싼기타 1 33.545 3.561 <NA> <NA> SPEAKER_00 <NA> <NA>
SPEAKER 싼기타_비싼기타 1 37.628 3.763 <NA> <NA> SPEAKER_00 <NA> <NA>
SPEAKER 싼기타_비싼기타 1 41.611 1.097 <NA> <NA> SPEAKER_00 <NA> <NA>
SPEAKER 싼기타_비싼기타 1 41.645 0.810 <NA> <NA> SPEAKER_01 <NA> <NA>
SPEAKER 싼기타_비싼기타 1 42.674 1.350 <NA> <NA> SPEAKER_01 <NA> <NA>
SPEAKER 싼기타_비싼기타 1 45.813 0.607 <NA> <NA> SPEAKER_00 <NA> <NA>
SPEAKER 싼기타_비싼기타 1 46.825 3.696 <NA> <NA> SPEAKER_00 <NA> <NA>
SPEAKER 싼기타_비싼기타 1 50.993 8.505 <NA> <NA> SPEAKER_00 <NA> <NA>
SPEAKER 싼기타_비싼기타 1 59.971 4.894 <NA> <NA> SPEAKER_00 <NA> <NA>
SPEAKER 싼기타_비싼기타 1 65.489 1.620 <NA> <NA> SPEAKER_00 <NA> <NA>
SPEAKER 싼기타_비싼기타 1 67.227 6.497 <NA> <NA> SPEAKER_01 <NA> <NA>
SPEAKER 싼기타_비싼기타 1 74.045 4.995 <NA> <NA> SPEAKER_01 <NA> <NA>
SPEAKER 싼기타_비싼기타 1 79.175 3.611 <NA> <NA> SPEAKER_01 <NA> <NA>
SPEAKER 싼기타_비싼기타 1 84.659 2.430 <NA> <NA> SPEAKER_00 <NA> <NA>
SPEAKER 싼기타_비싼기타 1 87.258 3.898 <NA> <NA> SPEAKER_00 <NA> <NA>
SPEAKER 싼기타_비싼기타 1 91.797 5.670 <NA> <NA> SPEAKER_00 <NA> <NA>
SPEAKER 싼기타_비싼기타 1 97.940 1.624 <NA> <NA> SPEAKER_00 <NA> <NA>
```

3. 문장과 화자 구분하기

- (실습) 화자 분리 모델로 시간대별 화자 구분하기

- 판다스를 활용해 데이터프레임 형태로 저장하기

1. 판다스의 .read_csv를 사용해 RTTM 파일의 데이터를 빈칸을 기준으로 나누어서 읽기

```
# RTTM을 CSV로 변환
import pandas as pd

rttm_path = "../audio/싼기타_비싼기타.rttm"

df_rttm = pd.read_csv(
    rttm_path,          # rttm 파일 경로
    sep=' ',           # 구분자는 띄어쓰기
    header=None,        # 헤더는 없음
    names=['type', 'file', 'chnl', 'start', 'duration', 'C1', 'C2', 'speaker_id', 'C3', 'C4']
)

display(df_rttm)
```

3. 문장과 화자 구분하기

- (실습) 화자 분리 모델로 시간대별 화자 구분하기

- 판다스를 활용해 데이터프레임 형태로 저장하기

1. 판다스의 `.read_csv`를 사용해 RTTM 파일의 데이터를 빈칸을 기준으로 나누어서 읽기

- 실행 결과

	type	file	chnl	start	duration	C1	C2	speaker_id	C3	C4
0	SPEAKER	싼기타_비싼기타	1	0.993	5.805	NaN	NaN	SPEAKER_01	NaN	NaN
1	SPEAKER	싼기타_비싼기타	1	7.405	3.983	NaN	NaN	SPEAKER_01	NaN	NaN
2	SPEAKER	싼기타_비싼기타	1	11.759	4.927	NaN	NaN	SPEAKER_01	NaN	NaN
3	SPEAKER	싼기타_비싼기타	1	17.210	10.665	NaN	NaN	SPEAKER_01	NaN	NaN
4	SPEAKER	싼기타_비싼기타	1	28.668	1.536	NaN	NaN	SPEAKER_01	NaN	NaN
...	...	...	...	...	...	...	...	...	...	...
83	SPEAKER	싼기타_비싼기타	1	414.481	2.970	NaN	NaN	SPEAKER_00	NaN	NaN
84	SPEAKER	싼기타_비싼기타	1	417.755	3.476	NaN	NaN	SPEAKER_01	NaN	NaN
85	SPEAKER	싼기타_비싼기타	1	423.644	0.776	NaN	NaN	SPEAKER_00	NaN	NaN
86	SPEAKER	싼기타_비싼기타	1	424.741	3.527	NaN	NaN	SPEAKER_00	NaN	NaN
87	SPEAKER	싼기타_비싼기타	1	428.504	0.844	NaN	NaN	SPEAKER_00	NaN	NaN

88 rows × 10 columns

3. 문장과 화자 구분하기

- (실습) 화자 분리 모델로 시간대별 화자 구분하기

- 판다스를 활용해 데이터프레임 형태로 저장하기

- 2. 발언이 끝난 시간 추가하기

```
# start + duration을 end로 변환  
df_rttm['end'] = df_rttm['start'] + df_rttm['duration']  
  
display(df_rttm)
```

	type	file	chnl	start	duration	C1	C2	speaker_id	C3	C4	end
0	SPEAKER	싼기타_비싼기타	1	0.993	5.805	NaN	NaN	SPEAKER_01	NaN	NaN	6.798
1	SPEAKER	싼기타_비싼기타	1	7.405	3.983	NaN	NaN	SPEAKER_01	NaN	NaN	11.388
2	SPEAKER	싼기타_비싼기타	1	11.759	4.927	NaN	NaN	SPEAKER_01	NaN	NaN	16.686
3	SPEAKER	싼기타_비싼기타	1	17.210	10.665	NaN	NaN	SPEAKER_01	NaN	NaN	27.875
4	SPEAKER	싼기타_비싼기타	1	28.668	1.536	NaN	NaN	SPEAKER_01	NaN	NaN	30.204
...	...	...	...	...	...	...	...	...	...	...	...
83	SPEAKER	싼기타_비싼기타	1	414.481	2.970	NaN	NaN	SPEAKER_00	NaN	NaN	417.451
84	SPEAKER	싼기타_비싼기타	1	417.755	3.476	NaN	NaN	SPEAKER_01	NaN	NaN	421.231
85	SPEAKER	싼기타_비싼기타	1	423.644	0.776	NaN	NaN	SPEAKER_00	NaN	NaN	424.420
86	SPEAKER	싼기타_비싼기타	1	424.741	3.527	NaN	NaN	SPEAKER_00	NaN	NaN	428.268
87	SPEAKER	싼기타_비싼기타	1	428.504	0.844	NaN	NaN	SPEAKER_00	NaN	NaN	429.348

88 rows × 11 columns

3. 문장과 화자 구분하기

- (실습) 화자 분리 모델로 시간대별 화자 구분하기

- 판다스를 활용해 데이터프레임 형태로 저장하기

3. 연속된 발화를 기록하기 위해 number 변수 추가하기

```
df_rttm["number"] = None # number 열 만들고 None으로 초기화
df_rttm.at[0, "number"] = 0
```

```
display(df_rttm)
```

	type	file	chnl	start	duration	C1	C2	speaker_id	C3	C4	end	number
0	SPEAKER	싼기타_비싼기타	1	0.993	5.805	NaN	NaN	SPEAKER_01	NaN	NaN	6.798	0
1	SPEAKER	싼기타_비싼기타	1	7.405	3.983	NaN	NaN	SPEAKER_01	NaN	NaN	11.388	None
2	SPEAKER	싼기타_비싼기타	1	11.759	4.927	NaN	NaN	SPEAKER_01	NaN	NaN	16.686	None
3	SPEAKER	싼기타_비싼기타	1	17.210	10.665	NaN	NaN	SPEAKER_01	NaN	NaN	27.875	None
4	SPEAKER	싼기타_비싼기타	1	28.668	1.536	NaN	NaN	SPEAKER_01	NaN	NaN	30.204	None
...	...	...	...	...	...	...	...	...	...	...	...	...
83	SPEAKER	싼기타_비싼기타	1	414.481	2.970	NaN	NaN	SPEAKER_00	NaN	NaN	417.451	None
84	SPEAKER	싼기타_비싼기타	1	417.755	3.476	NaN	NaN	SPEAKER_01	NaN	NaN	421.231	None
85	SPEAKER	싼기타_비싼기타	1	423.644	0.776	NaN	NaN	SPEAKER_00	NaN	NaN	424.420	None
86	SPEAKER	싼기타_비싼기타	1	424.741	3.527	NaN	NaN	SPEAKER_00	NaN	NaN	428.268	None
87	SPEAKER	싼기타_비싼기타	1	428.504	0.844	NaN	NaN	SPEAKER_00	NaN	NaN	429.348	None

88 rows × 12 columns

3. 문장과 화자 구분하기

- (실습) 화자 분리 모델로 시간대별 화자 구분하기

- 판다스를 활용해 데이터프레임 형태로 저장하기

4. 화자 번호 매기기

```
for i in range(1, len(df_rttm)):
    if df_rttm.at[i, "speaker_id"] != df_rttm.at[i-1, "speaker_id"]:
        df_rttm.at[i, "number"] = df_rttm.at[i-1, "number"] + 1
    else:
        df_rttm.at[i, "number"] = df_rttm.at[i-1, "number"]

display(df_rttm.head(10))
```

3. 문장과 화자 구분하기

- (실습) 화자 분리 모델로 시간대별 화자 구분하기

- 판다스를 활용해 데이터프레임 형태로 저장하기

- 4. 화자 번호 매기기

- 실행 결과

	type	file	chnl	start	duration	C1	C2	speaker_id	C3	C4	end	number
0	SPEAKER	싼기타_비싼기타	1	0.993	5.805	NaN	NaN	SPEAKER_01	NaN	NaN	6.798	0
1	SPEAKER	싼기타_비싼기타	1	7.405	3.983	NaN	NaN	SPEAKER_01	NaN	NaN	11.388	0
2	SPEAKER	싼기타_비싼기타	1	11.759	4.927	NaN	NaN	SPEAKER_01	NaN	NaN	16.686	0
3	SPEAKER	싼기타_비싼기타	1	17.210	10.665	NaN	NaN	SPEAKER_01	NaN	NaN	27.875	0
4	SPEAKER	싼기타_비싼기타	1	28.668	1.536	NaN	NaN	SPEAKER_01	NaN	NaN	30.204	0
5	SPEAKER	싼기타_비싼기타	1	32.414	0.759	NaN	NaN	SPEAKER_00	NaN	NaN	33.173	1
6	SPEAKER	싼기타_비싼기타	1	33.545	3.561	NaN	NaN	SPEAKER_00	NaN	NaN	37.106	1
7	SPEAKER	싼기타_비싼기타	1	37.628	3.763	NaN	NaN	SPEAKER_00	NaN	NaN	41.391	1
8	SPEAKER	싼기타_비싼기타	1	41.611	1.097	NaN	NaN	SPEAKER_00	NaN	NaN	42.708	1
9	SPEAKER	싼기타_비싼기타	1	41.645	0.810	NaN	NaN	SPEAKER_01	NaN	NaN	42.455	2

3. 문장과 화자 구분하기

- (실습) 화자 분리 모델로 시간대별 화자 구분하기

- 판다스를 활용해 데이터프레임 형태로 저장하기

- 5. 같은 화자끼리 묶어서 정리하기

```
df_rttm_grouped = df_rttm.groupby("number").agg(  
    start=pd.NamedAgg(column='start', aggfunc='min'),  
    end=pd.NamedAgg(column='end', aggfunc='max'),  
    speaker_id=pd.NamedAgg(column='speaker_id', aggfunc='first')  
)  
  
display(df_rttm_grouped)
```

	start	end	speaker_id
number			
0	0.993	30.204	SPEAKER_01
1	32.414	42.708	SPEAKER_00
2	41.645	44.024	SPEAKER_01
3	45.813	67.109	SPEAKER_00
4	67.227	82.786	SPEAKER_01
5	84.659	102.564	SPEAKER_00
6	103.492	117.532	SPEAKER_01
7	119.759	138.676	SPEAKER_00
8	139.351	168.967	SPEAKER_01
9	170.907	192.321	SPEAKER_00
10	192.322	193.689	SPEAKER_01

3. 문장과 화자 구분하기

- (실습) 화자 분리 모델로 시간대별 화자 구분하기

- 판다스를 활용해 데이터프레임 형태로 저장하기

- 6. 발화 시간 추가하고 인덱스 제거하기

```
df_rttm_grouped["duration"] = df_rttm_grouped["end"] - df_rttm_grouped["start"]
df_rttm_grouped = df_rttm_grouped.reset_index(drop=True)
display(df_rttm_grouped)
```

	start	end	speaker_id	duration
0	0.993	30.204	SPEAKER_01	29.211
1	32.414	42.708	SPEAKER_00	10.294
2	41.645	44.024	SPEAKER_01	2.379
3	45.813	67.109	SPEAKER_00	21.296
4	67.227	82.786	SPEAKER_01	15.559
5	84.659	102.564	SPEAKER_00	17.905
6	103.492	117.532	SPEAKER_01	14.040
7	119.759	138.676	SPEAKER_00	18.917
8	139.351	168.967	SPEAKER_01	29.616
9	170.907	192.321	SPEAKER_00	21.414
10	192.322	193.689	SPEAKER_01	1.367
11	193.760	193.503	SPEAKER_00	0.257

3. 문장과 화자 구분하기

- (실습) 화자 분리 모델로 시간대별 화자 구분하기

- 판다스를 활용해 데이터프레임 형태로 저장하기

7. 화자 분리 결과를 CSV 파일로 저장하기

```
df_rttm_grouped.to_csv(  
    "../audio/싼기타_비싼기타_rttm.csv",  
    sep=',',  
    index=False  
)
```

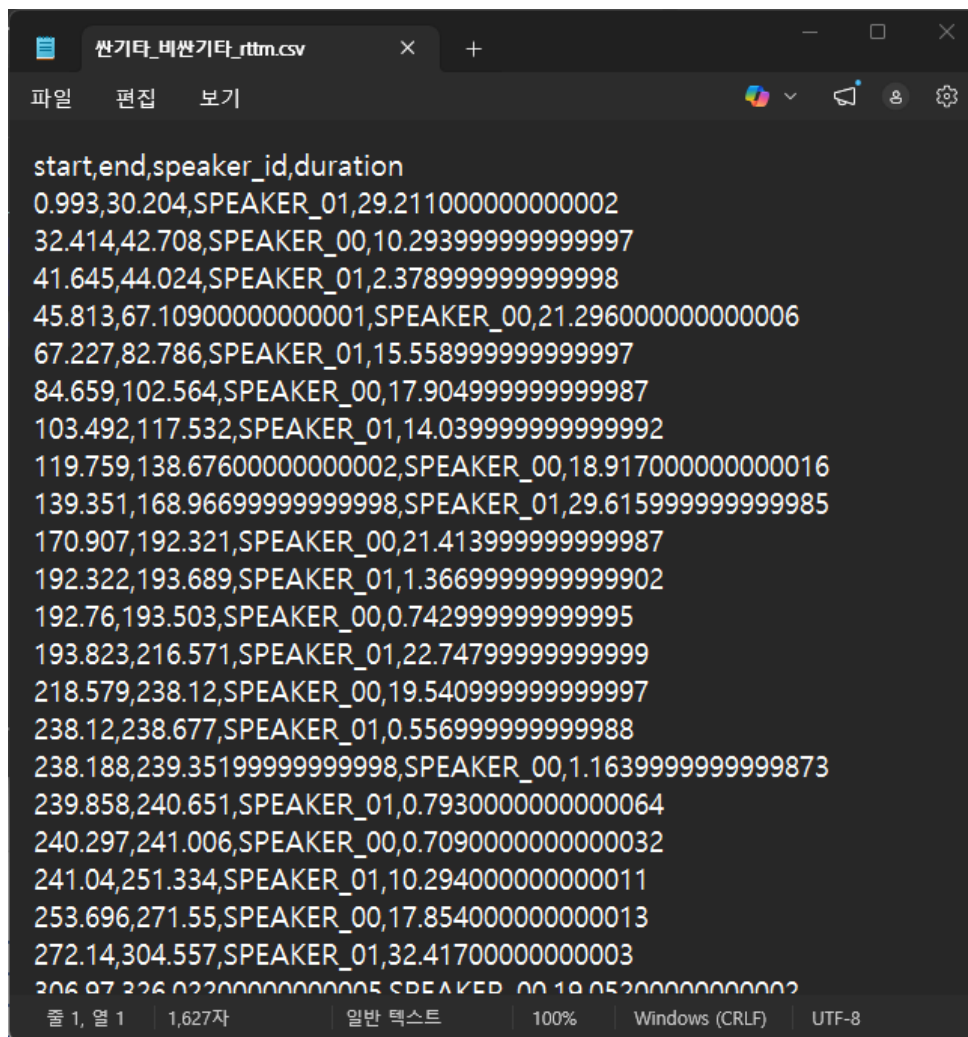
3. 문장과 화자 구분하기

- (실습) 화자 분리 모델로 시간대별 화자 구분하기

- 판다스를 활용해 데이터프레임 형태로 저장하기

- 7. 화자 분리 결과를 CSV 파일로 저장하기

- 실행 결과



```
start,end,speaker_id,duration
0.993,30.204,SPEAKER_01,29.211000000000002
32.414,42.708,SPEAKER_00,10.293999999999997
41.645,44.024,SPEAKER_01,2.378999999999998
45.813,67.109000000000001,SPEAKER_00,21.296000000000006
67.227,82.786,SPEAKER_01,15.558999999999997
84.659,102.564,SPEAKER_00,17.904999999999987
103.492,117.532,SPEAKER_01,14.039999999999992
119.759,138.676000000000002,SPEAKER_00,18.917000000000016
139.351,168.96699999999998,SPEAKER_01,29.615999999999985
170.907,192.321,SPEAKER_00,21.413999999999987
192.322,193.689,SPEAKER_01,1.3669999999999902
192.76,193.503,SPEAKER_00,0.742999999999995
193.823,216.571,SPEAKER_01,22.747999999999999
218.579,238.12,SPEAKER_00,19.540999999999997
238.12,238.677,SPEAKER_01,0.556999999999998
238.188,239.35199999999998,SPEAKER_00,1.1639999999999873
239.858,240.651,SPEAKER_01,0.79300000000000064
240.297,241.006,SPEAKER_00,0.70900000000000032
241.04,251.334,SPEAKER_01,10.294000000000011
253.696,271.55,SPEAKER_00,17.854000000000013
272.14,304.557,SPEAKER_01,32.417000000000003
306.87,326.022000000000005,SPEAKER_00,19.052000000000002
```

3. 문장과 화자 구분하기

- (실습) 판다스로 문장 분석하고 화자 매칭하기
 - 로컬에 설치한 위스퍼 모델을 활용해 받아쓰기 함수 만들기
 1. whisper_stt.py 파일 만들고, whisper_stt 함수 정의

```
... 생략 ...

def whisper_stt(
    audio_file_path: str,
    output_file_path: str = "./output.csv"
):
    device = "cuda:0" if torch.cuda.is_available() else "cpu"
    torch_dtype = torch.float16 if torch.cuda.is_available() else
torch.float32
    model_id = "openai/whisper-large-v3-turbo"

    model = AutoModelForSpeechSeq2Seq.from_pretrained(
        model_id, torch_dtype=torch_dtype,
        low_cpu_mem_usage=True,
        use_safetensors=True
    )
    model.to(device)

... 생략 ...
```

3. 문장과 화자 구분하기

- (실습) 판다스로 문장 분석하고 화자 매칭하기
 - 로컬에 설치한 위스퍼 모델을 활용해 받아쓰기 함수 만들기

2. whisper_to_dataframe 함수 만들기

```
def whisper_to_dataframe(result, output_file_path):
    start_end_text = []

    for chunk in result["chunks"]:
        start = chunk["timestamp"][0]
        end = chunk["timestamp"][1]
        text = chunk["text"].strip()
        start_end_text.append([start, end, text])
    df = pd.DataFrame(start_end_text, columns=["start", "end", "text"])
    df.to_csv(output_file_path, index=False, sep="|")

    return df
```

3. 문장과 화자 구분하기

- (실습) 판다스로 문장 분석하고 화자 매칭하기
 - 로컬에 설치한 위스퍼 모델을 활용해 받아쓰기 함수 만들기

3. 메인 영역을 선언하고 whisper_stt 함수 실행 후 결과 출력

```
if __name__ == "__main__":  
    audio_file_path = "./audio/싼기타_비싼기타.mp3"           # 원본 오디오 파일  
    stt_output_file_path = "./audio/싼기타_비싼기타.csv"      # STT 결과 파일  
  
    result, df = whisper_stt(  
        audio_file_path,  
        stt_output_file_path  
    )  
    print(df)
```

3. 문장과 화자 구분하기

- (실습) 판다스로 문장 분석하고 화자 매칭하기

- 로컬에 설치한 위스퍼 모델을 활용해 받아쓰기 함수 만들기

3. 결과 출력 및 저장

	start	end	text
0	1.00	4.00	지금부터 저랑 역할극을 합시다.
1	4.00	7.00	역할극을 스탠드인 코미디 스타일로 할 건데
2	7.00	10.08	토론을 하면서 자연스럽게 대화하는 형식으로 하면서
3	10.08	11.22	코미디를 진행해봅시다.
4	11.82	16.04	그래서 좀 재밌고 자연스럽게 유머러스하게 저랑 대화를 하시면 돼요.
..	...	...	...
110	398.00	406.50	아니요, 전혀 화나지 않았어요. 편하게 이야기 나누는 중이니까 걱정
111	407.24	408.14	화난 것 같은데?
112	414.00	414.54	네, 정말 괜찮습니다. 즐겁게 대화 나누고 있었어요.
113	417.00	423.00	계속해서 이하고 나누고 싶으시면 편하게 말씀해주세요. 아니요. 화나셨는데
114	423.00	428.96	알겠습니다. 언제든 다시 이야기 나누고 싶으실 때 편하게 말씀해 주세요.

```
start|end|text
1.0|4.0|지금부터 저랑 역할극을 합시다.
4.0|7.0|역할극을 스탠드인 코미디 스타일로 할 건데
7.0|10.08|토론을 하면서 자연스럽게 대화하는 형식으로 하면서
10.08|11.22|코미디를 진행해봅시다.
11.82|16.04|그래서 좀 재밌고 자연스럽게 유머러스하게 저랑 대화를 하시면 돼요.
16.12|16.62|자연스럽게.
17.32|21.4|그리고 주제는 쌍기타로 전기기타를 시작하는 게 좋으나
21.4|24.0|아니면 비싼 기타로 전기기타를 시작하는 게 좋으나
24.0|26.0|이거를 입장을 나눠가지고
26.0|28.0|저랑 토론해보면 좋을 것 같아요.
28.0|30.0|어, 두 중에 어떤 역할 맡으실래요?
41.0|42.0|좋습니다. 그럼 제가 싼 기타로 시작하는 게 좋다는 입장을 맡아볼게요. 그럼 성형님
은 비싼 기타로 시작하는 게 좋다는 입장이시죠? 네 맞아요. 준비 되셨나요?
42.0|44.0|네, 됐어요. 시작하시죠.
44.0|50.24|좋아요. 먼저 3기타로 시작하는 게 좋은 이유를 말씀드리겠습니다.
51.06|54.9|초보자일 때는 실수도 많고 기타에 익숙해지는 과정이 필요하니까
54.9|60.0|비싼 기타보다는 부담 없이 연습할 수 있는 싼 기타가 좋다고 봅니다.
60.0|65.0|무엇보다 실력이 향상되면 그때 좋은 기타로 업그레이드 하는 것도 나쁘지 않죠?
65.0|67.0|성형님은 어떻게 생각하세요?
67.0|69.82|아, 저는 지금 말에 어폐가 있다고 생각해요.
69.92|72.2|왜냐하면 어차피 지금 비싼 기타로
72.2|72.5|이 주제에 사 새가오 하느 거잖아오
```

3. 문장과 화자 구분하기

- (실습) 판다스로 문장 분석하고 화자 매칭하기

- 시간대별로 화자를 구분하는 함수 만들기

1. 앞서 만들었던 화자 분리 기능을 speaker_diarizaion 함수로 만들기

```
... 생략 ...

def speaker_diarization(
    audio_file_path: str,
    output_rttm_file_path: str,
    output_csv_file_path: str
):
    pipeline = Pipeline.from_pretrained(
        "pyannote/speaker-diarization-3.1",
        use_auth_token=access_tokens
    )

    # cuda가 사용 가능한 경우 cuda를 사용하도록 설정
    if torch.cuda.is_available():
        pipeline.to(torch.device("cuda"))
        print('cuda is available')
    else:
        print('cuda is not available')
... 생략 ...
```

3. 문장과 화자 구분하기

- (실습) 판다스로 문장 분석하고 화자 매칭하기

- 시간대별로 화자를 구분하는 함수 만들기

2. 파이썬 코드의 메인 부분도 이에 맞게 수정

```
if __name__ == "__main__":
    audio_file_path = "./audio/싼기타_비싼기타.mp3"          # 원본 오디오 파일
    rttm_file_path = "./audio/싼기타_비싼기타.rttm"          # 화자 분리 원본 파일
    rttm_csv_file_path = "./audio/싼기타_비싼기타_rttm.csv"  # 화자 분리 CSV 파일
    final_csv_file_path = "./audio/싼기타_비싼기타_final.csv" # 최종 결과 파일

    df_rttm = speaker_diarization(
        audio_file_path,
        rttm_file_path,
        rttm_csv_file_path
    )

    print(df_rttm)
```

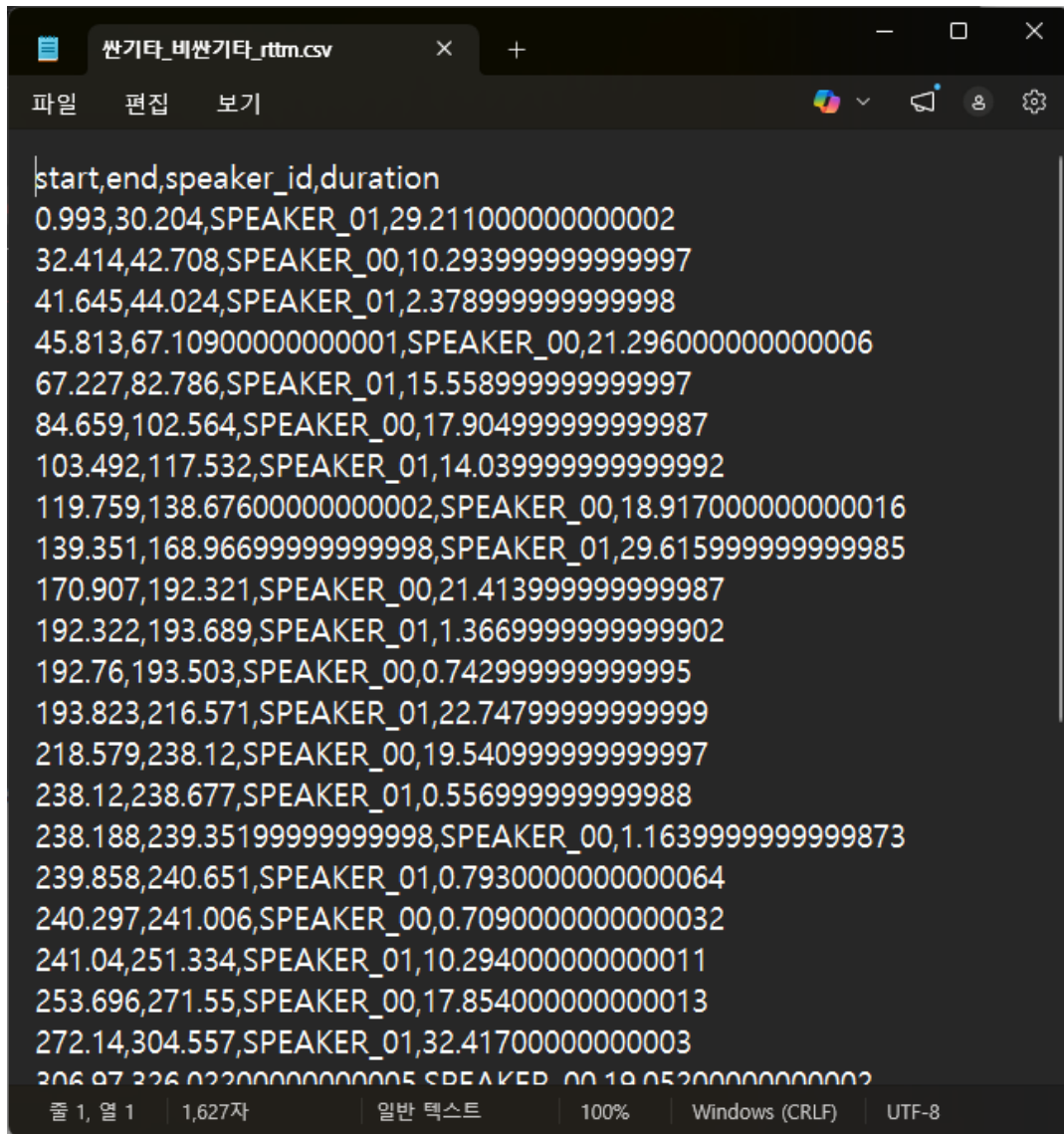
3. 문장과 화자 구분하기

- (실습) 판다스로 문장 분석하고 화자 매칭하기

- 시간대별로 화자를 구분하는 함수 만들기

- 3. 파이썬 코드의 메인 부분도 이에 맞게 수정

- 지정 경로에 파일이 잘 생성되어 있는지 확인



```
start,end,speaker_id,duration
0.993,30.204,SPEAKER_01,29.211000000000002
32.414,42.708,SPEAKER_00,10.293999999999997
41.645,44.024,SPEAKER_01,2.3789999999999998
45.813,67.109000000000001,SPEAKER_00,21.296000000000006
67.227,82.786,SPEAKER_01,15.558999999999997
84.659,102.564,SPEAKER_00,17.904999999999998
103.492,117.532,SPEAKER_01,14.039999999999992
119.759,138.67600000000002,SPEAKER_00,18.917000000000016
139.351,168.96699999999998,SPEAKER_01,29.615999999999985
170.907,192.321,SPEAKER_00,21.413999999999987
192.322,193.689,SPEAKER_01,1.3669999999999902
192.76,193.503,SPEAKER_00,0.7429999999999995
193.823,216.571,SPEAKER_01,22.747999999999999
218.579,238.12,SPEAKER_00,19.540999999999997
238.12,238.677,SPEAKER_01,0.5569999999999988
238.188,239.35199999999998,SPEAKER_00,1.1639999999999873
239.858,240.651,SPEAKER_01,0.79300000000000064
240.297,241.006,SPEAKER_00,0.70900000000000032
241.04,251.334,SPEAKER_01,10.2940000000000011
253.696,271.55,SPEAKER_00,17.8540000000000013
272.14,304.557,SPEAKER_01,32.417000000000003
306.97,326.02200000000005,SPEAKER_00,19.052000000000002
```

줄 1, 열 1 | 1,627자 | 일반 텍스트 | 100% | Windows (CRLF) | UTF-8

3. 문장과 화자 구분하기

- (실습) 판다스로 문장 분석하고 화자 매칭하기

- STT 결과 파일과 화자 분리 결과 파일(RTTM) 결합하기

1. 화자별로 발언을 할당하기 위한 stt_to_rttm 함수 만들기

```
... 생략 ...

def stt_to_rttm(
    audio_file_path: str, stt_output_file_path: str,
    rttm_file_path: str, rttm_csv_file_path: str,
    final_output_csv_file_path: str
):
    result, df_stt = whisper_stt(
        audio_file_path,
        stt_output_file_path
    )

    df_rttm = speaker_diarization(
        audio_file_path,
        rttm_file_path,
        rttm_csv_file_path
    )
    생략 ...
```

3. 문장과 화자 구분하기

- (실습) 판다스로 문장 분석하고 화자 매칭하기

- STT 결과 파일과 화자 분리 결과 파일(RTTM) 결합하기

2. 기존에 만든 함수들은 모두 주석으로 처리하고 최종 결과를 저장할 파일 경로 추가

```
if __name__ == "__main__":
    audio_file_path = "./audio/싼기타_비싼기타.mp3"           # 원본 오디오 파일
    stt_output_file_path = "./audio/싼기타_비싼기타.csv"      # STT 결과 파일
    rttm_file_path = "./audio/싼기타_비싼기타.rttm"          # 화자 분리 원본 파일
    rttm_csv_file_path = "./audio/싼기타_비싼기타_rttm.csv"   # 화자 분리 CSV 파일
    final_csv_file_path = "./audio/싼기타_비싼기타_final.csv" # 최종 결과 파일

    df_rttm = stt_to_rttm(
        audio_file_path,
        stt_output_file_path,
        rttm_file_path,
        rttm_csv_file_path,
        final_csv_file_path
    )

    print(df_rttm)
```

3. 문장과 화자 구분하기

- (실습) 판다스로 문장 분석하고 화자 매칭하기

- STT 결과 파일과 화자 분리 결과 파일(RTTM) 결합하기

2. 기존에 만든 함수들은 모두 주석으로 처리하고 최종 결과를 저장할 파일 경로 추가

number	start	end	speaker_id	duration	text
0	0.993	30.204	SPEAKER_01	29.211	지금부터 저랑 역할극을 합시다.\n역할극을 스탠드인 코미디 스타일로 할 건데\n토론...
1	32.414	42.708	SPEAKER_00	10.294	좋습니다. 그럼 제가 썸 기타로 시작하는 게 좋다는 입장을 말아볼게요. 그럼 성형님...
2	41.645	44.024	SPEAKER_01	2.379	네, 됐어요. 시작하시죠.\n
3	45.813	67.109	SPEAKER_00	21.296	좋아요. 먼저 3기타로 시작하는 게 좋은 이유를 말씀드리겠습니다.\n초보자일 때는 ...
4	67.227	82.786	SPEAKER_01	15.559	아, 저는 지금 말에 어폐가 있다고 생각해요.\n왜냐하면 어차피 지금 비싼 기타로\...
5	84.659	102.564	SPEAKER_00	17.905	그런데 비싼 기타로 시작하면\n혹시라도 흠집이 나거나 실수할 때 부담이 더 크지 않...
6	103.492	117.532	SPEAKER_01	14.040	썸 기타를 뭐하러 또 샅니까?\n그리고 기타 실력이랑 흠집이랑은 아무 상관이 없어요...
7	119.759	138.676	SPEAKER_00	18.917	하하! 기타를 망치로 치진 않지만 그래도 초보자들은 실수도 많고 조심스럽게 다루기 ...
8	139.351	168.967	SPEAKER_01	29.616	충분하죠. 아 맞아요. 그것도 맞는 말이에요. 쌍기타도 요새 품질이\n많이 좋아서\...
9	170.907	192.321	SPEAKER_00	21.414	하하\n역시 좋은 기타를 사면 책임감도\n더 커진다는 말씀이시군요. 그래도 비싼 기...
10	192.322	193.689	SPEAKER_01	1.367	
11	192.760	193.503	SPEAKER_00	0.743	
12	193.823	216.571	SPEAKER_01	22.748	쌍기타로 시작해서 부담없이 연습하고 정말 기타에 흥미가 생기면 그때 비슷하게 아니에...
13	218.579	238.120	SPEAKER_00	19.541	하하 비싼 기타가\n일종의 동기부여가 된다는\n말씀이시군요. 하지만 반대로 비싼 기...
14	238.120	238.677	SPEAKER_01	0.557	

4. 회의록을 정리하는 AI 서기 완성하기

- (실습) 전체 회의 내용 요약하기

- 앞서 분석한 결과를 바탕으로 회의록을 MS워드로 완성해 봄
- 회의록에는 다음 2가지 내용을 담으려고 함
 - 전체 회의 내용 요약
 - 화자별 발언 내용을 순서대로 정리

1. 단계별로 진행하기 위해 주피터 노트북 파일 생성 - summarize_and_correct.ipynb

- 앞서 만들었던 CSV 파일에서 SPEAKER_00, SPEAKER_01로 표시되었던 부분에 화자의 이름을 넣고 GPT에게 회의 내용 요약을 요청함

4. 회의록을 정리하는 AI 서기 완성하기

- (실습) 전체 회의 내용 요약하기

2. 기존에 만들었던 회의록 CSV 파일을 열어 df_rttm 변수에 담기

```
import pandas as pd

meeting_note_csv_path = "../audio/싼기타_비싼기타_final.csv"
df_rttm = pd.read_csv(meeting_note_csv_path, sep='|')
```

```
display(df_rttm)
```

	start	end	speaker_id	duration	text
0	0.993	30.204	SPEAKER_01	29.211	지금부터 저랑 역할극을 합시다.\n역할극을 스탠드인 코미디 스타일로 할 건데\n토론...
1	32.414	42.708	SPEAKER_00	10.294	좋습니다. 그럼 제가 싼 기타로 시작하는 게 좋다는 입장을 말아볼게요. 그럼 성형님...
2	41.645	44.024	SPEAKER_01	2.379	네, 됐어요. 시작하시죠.\n
3	45.813	67.109	SPEAKER_00	21.296	좋아요. 먼저 3기타로 시작하는 게 좋은 이유를 말씀드리겠습니다.\n초보자일 때는 ...
4	67.227	82.786	SPEAKER_01	15.559	아, 저는 지금 말에 어폐가 있다고 생각해요.\n왜냐하면 어차피 지금 비싼 기타로\...
5	84.659	102.564	SPEAKER_00	17.905	그런데 비싼 기타로 시작하면\n혹시라도 흠집이 나거나 실수할 때 부담이 더 크지 않...
6	103.492	117.532	SPEAKER_01	14.040	싼 기타를 뭐하러 또 샅까?\n그리고 기타 실력이랑 흠집이랑은 아무 상관이 없어요...
7	119.759	138.676	SPEAKER_00	18.917	하하! 기타를 망치로 치진 않지만 그래도 초보자들은 실수도 많고 조심스럽게 다루기 ...
8	139.351	168.967	SPEAKER_01	29.616	충분하죠. 아 맞아요. 그것도 맞는 말이에요. 싹기타도 요새 품질이\n많이 좋아서\...
9	170.907	192.321	SPEAKER_00	21.414	하하\n역시 좋은 기타를 사면 책임감도\n더 커진다는 말씀이시군요. 그래도 비싼 기...
10	192.322	193.689	SPEAKER_01	1.367	NaN
11	193.700	193.503	SPEAKER_00	0.743	NaN

4. 회의록을 정리하는 AI 서기 완성하기

- (실습) 전체 회의 내용 요약하기

3. SPEAKER_00과 SPEAKER_01로 표시된 부분을 활용해 실제 화자의 이름 넣기

```
# 이름 넣기
name_dict = {
    "SPEAKER_00": "AI",
    "SPEAKER_01": "홍길동",
}

df_rttm["name"] = df_rttm["speaker_id"].apply(lambda x: name_dict[x])

display(df_rttm)
```

	start	end	speaker_id	duration	text	name
0	0.993	30.204	SPEAKER_01	29.211	지금부터 저랑 역할극을 합시다.\n역할극을 스탠드인 코미디 스타일로 할 건데\n토론...	홍길동
1	32.414	42.708	SPEAKER_00	10.294	좋습니다. 그럼 제가 썸 기타로 시작하는 게 좋다는 입장을 알아볼게요. 그럼 성형님...	AI
2	41.645	44.024	SPEAKER_01	2.379	네, 됐어요. 시작하시죠.\n	홍길동
3	45.813	67.109	SPEAKER_00	21.296	좋아요. 먼저 3기타로 시작하는 게 좋은 이유를 말씀드리겠습니다.\n초보자일 때는 ...	AI
4	67.227	82.786	SPEAKER_01	15.559	아, 저는 지금 말에 어폐가 있다고 생각해요.\n왜냐하면 어차피 지금 비싼 기타로...	홍길동
5	84.659	102.564	SPEAKER_00	17.905	그런데 비싼 기타로 시작하면\n혹시라도 흠집이 나거나 실수할 때 부담이 더 크지 않...	AI
6	103.492	117.532	SPEAKER_01	14.040	싼 기타를 뭐하러 또 샅니까?\n그리고 기타 실력이랑 흠집이랑은 아무 상관이 없어요...	홍길동
7	119.759	138.676	SPEAKER_00	18.917	하하! 기타를 망치로 치진 않지만 그래도 초보자들은 실수도 많고 조심스럽게 다루기 ...	AI
8	139.351	168.967	SPEAKER_01	29.616	충분하죠. 아 맞아요. 그것도 맞는 말이에요. 쌍기타도 요새 품질이\n많이 좋아서...	홍길동

4. 회의록을 정리하는 AI 서기 완성하기

- (실습) 전체 회의 내용 요약하기

4. df_rttm에서 필요한 정보만 뽑기

```
meeting_note_txt = df_rttm[['start', 'end', 'name', 'text']].to_json(orient='records', force_ascii=False)
print(meeting_note_txt)
```

```
[{"start":0.993,"end":30.204,"name":"홍길동","text":"지금부터 저랑 역할극을 합시다.\n역할극을 스탠드인 코미디 스타일로 할 건데\n토론을 하면서 자연스럽게 대화하는 형식으로 하면서\n코미디를 진행해봅시다.\n그래서 좀 재밌고 자연스럽게 유머러스하게 저랑 대화를 하시면 돼요.\n자연스럽게.\n그리고 주제는 쌍기타로 전기기타를 시작하는 게 좋냐\n아니면 비싼 기타로 전기기타를 시작하는 게 좋냐\n이거를 입장을 나눠가지고\n저랑 토론해보면 좋을 것 같아요.\n어, 두 중에 어떤 역할 맡으실래요?\n"}, {"start":32.414,"end":42.708,"name":"AI","text":"좋습니다. 그럼 제가 싼 기타로 시작하는 게 좋다는 입장을 맡아볼게요. 그럼 성형님은 비싼 기타로 시작하는 게 좋다는 입장이시죠? 네 맞아요. 준비 되셨나요?\n"}, {"start":41.645,"end":44.024,"name":"홍길동","text":"네, 됐어요. 시작하시죠.\n"}, {"start":45.813,"end":67.109,"name":"AI","text":"좋아요. 먼저 3기타로 시작하는 게 좋은 이유를 말씀드리겠습니다.\n초보자일 때는 실수도 많고 기타에 익숙해지는 과정이 필요하니까\n비싼 기타보다는 부담 없이 연습할 수 있는 싼 기타가 좋다고 봅니다.\n무엇보다 실력이 향상되면 그때 좋은 기타로 업그레이드 하는 것도 나쁘지 않죠?\n성형님은 어떻게 생각하세요?\n"}, {"start":67.227,"end":82.786,"name":"홍길동","text":"아, 저는 지금 말에 어폐가 있다고 생각해요.\n왜냐하면 어차피 지금 비싼 기타로\n나중에 살 생각을 하는 거잖아요.\n그럼 어차피 비싼 기타 사고 쌍기타 사게 되는데 그런 이중 이중 지출을 하지 말고 처음부터 비싼 기타로 가면은 이중지출을 하지 말고 처음부터 비싼 기타로 가면 이중지출을 막을 수가 있습니다.\n"}, {"start":84.659,"end":102.564,"name":"AI","text":"그런데 비싼 기타로 시작하면\n혹시라도 흠집이 나거나 실수할 때 부담이 더 크지 않을까요?\n초보 때는 실습을 많이 하니까 쌍기타로 마음 편히 연습할 수 있는 게 더 좋다고 생각해요.\n그리고 기타 연주에 익숙해지면 그때 비교할 수 있는 게 더 좋다고 생각해요.\n그리고"}]
```

(... 생략 ...)

```
[{"start":423.644,"end":429.348,"name":"AI","text":"알겠습니다. 언제든지 다시 이야기 나누고 싶으실 때 편하게 말씀해 주세요. 감사합니다.\n"}]
```

4. 회의록을 정리하는 AI 서기 완성하기

- (실습) 전체 회의 내용 요약하기

5. GPT에게 회의록을 작성해 달라고 프롬프트 작성

```
system_prompt = f'''
너는 회의 내용을 요약하는 봇이다. 아래 회의록을 읽고, 주요 내용을 요약하라.
결과는 마크다운 형식으로 작성한다.

아래 형식에 맞추어 작성하라.

# 회의 제목

## 주요 내용

## 참석자별 입장

## 결정 사항

===== 이하 회의록 =====

{ meeting_note_txt }
'''
```

4. 회의록을 정리하는 AI 서기 완성하기

- (실습) 전체 회의 내용 요약하기

6. GPT에게 요청하기 전에 오픈AI의 API client 선언

```
from openai import OpenAI
from dotenv import load_dotenv
import os

load_dotenv()
api_key = os.getenv('OPENAI_API_KEY')
client = OpenAI(api_key=api_key)
```

4. 회의록을 정리하는 AI 서기 완성하기

- (실습) 전체 회의 내용 요약하기

7. GPT 클라이언트를 이용해 프롬프트 보내기

```
# OpenAI API를 사용해 요약 결과 생성
response = client.chat.completions.create(
    model='gpt-5-nano',
    messages=[
        {"role": "system", "content": system_prompt},
    ]
)

# (4) 요약 결과를 파일로 저장
summary = response.choices[0].message.content
summary = summary.strip() # 좌우 공백 제거

print(summary)
```

4. 회의록을 정리하는 AI 서기 완성하기

● (실습) 전체 회의 내용 요약하기

7. GPT 클라이언트를 이용해 프롬프트 보내기

회의 제목

쌍기타 시작 시기 선택: 싼 기타로 시작 vs 비싼 기타로 시작

주요 내용

- 회의 형태: 역할극 형식의 스탠드인 코미디 토론으로 주제를 다룸. 주제는 쌍기타(저가) 시작 vs 비싼 기타 시작의 장단점 비교.
- 핵심 논점
 - 저가 기타로 시작하는 쪽의 주장: 초보자 부담이 적고, 실력 향상 시점에 맞춰 고가 기타로 업그레이드하는 전략이 유효. 저가 기타도 최근 품질이 좋아 연습에 충분하다는 점 강조.
 - 고가 기타로 시작하는 쪽의 주장: 처음부터 고가 기타를 사면 이중지출을 막을 수 있고, 동기부여가 커져 꾸준한 연습으로 이어진다는 점 강조. 그러나 초보 시 실수나 흠집에 대한 부담도 큼.
- 중간 입장 및 합의 포인트
 - 두 입장 모두 기타의 품질 차이가 연습에 영향을 줄 수 있음.
 - 결국 중요한 것은 개인의 상황과 열정, 꾸준한 연습이라는 결론에 다가감.
- 분위기 및 결론: 토론은 재미있고 자연스러운 대화를 지향했고, 특정 승패보다는 기타에 대한 열정을 유지하는 것이 핵심 목표로 남음.

참석자별 입장

- 홍길동
 - 주된 입장: 비싼 기타로 시작하는 것이 좋다고 주장.
 - 근거 요지:
 - 초기 비용을 들여야 이중지출(쌍기타와 고가 기타의 중복 구매)을 피할 수 있음.
 - 고가 기타로 시작하면 초기 실력 차이에도 불구하고 지속적으로 연습할 동기 부여가 강해짐.
 - 저가 기타로 시작하면 고가 기타의 경험을 못해 “맛”을 모르는 상황이 생길 수 있음.
 - 추가 논점: 품질 차이가 장기적으로 학습 효과를 높인다고 봄.
- AI (역할 담당)
 - ...
 - 최적의 선택은 개인의 상황과 성향에 따라 다름.
 - 초보의 부담과 동기부여를 모두 고려해, 각자의 상황에 맞춰 선택하는 것이 최선이다.
 - 중요한 것은 꾸준한 연습과 기타에 대한 열정이며, 필요 시 업그레이드 계획을 세우는 것이 바람직하다는 합의에 가까운 결론이 도출됨.
 - 추가 메모: 대화는 즐겁고 유머러스한 분위기로 진행되었으며, 토론의 목적은 서로의 입장을 이해하고 기타에 대한 열정을 나누는 데 있었음.

4. 회의록을 정리하는 AI 서기 완성하기

● (실습) 전체 회의 내용 요약하기

8. 회의록을 guitar_summary.md로 저장

```
with open('../audio/guitar_summary.md', 'w', encoding='utf-8') as f:  
    f.write(summary)
```

The screenshot displays a Jupyter Notebook interface with a file explorer on the left showing 'guitar_summary.md'. The main area shows the content of this file, which is a meeting summary. To the right, a 'Preview guitar_summary.md' window shows the rendered Markdown format.

회의 제목

쌍기타 시작 시기 선택: 썬 기타로 시작 vs 비싼 기타로 시작

주요 내용

- 회의 형태: 역할극 형식의 스탠드인 코미디 토론으로 주제를 다룸. 주제는 쌍기타(저가) 시작 vs 비싼 기타 시작의 장단점 비교.
- 핵심 논점
 - 저가 기타로 시작하는 쪽의 주장: 초보자 부담이 적고, 실력 향상 시점에 맞춰 고가 기타로 업그레이드하는 전략이 유효. 저가 기타도 최근 품질이 좋아 연습에 충분하다는 점 강조.
 - 고가 기타로 시작하는 쪽의 주장: 처음부터 고가 기타를 사면 이중지출을 막을 수 있고, 동기부여가 커져 꾸준한 연습으로 이어진다는 점 강조. 그러나 초보 시 실수나 흠집에 대한 부담도 큼.
- 중간 입장 및 합의 포인트
 - 두 입장 모두 기타의 품질 차이가 연습에 영향을 줄 수 있음.
 - 결국 중요한 것은 개인의 상황과 열정, 꾸준한 연습이라는 결론에 다감.
- 분위기 및 결론: 토론은 재미있고 자연스러운 대화를 지향했고, 특정 승패보다는 기타에 대한 열정을 유지하는 것이 핵심 목표로 남음.

참석자별 입장

- 홍길동
 - 주된 입장: 비싼 기타로 시작하는 것이 좋다고 주장.
 - 근거 요지:
 - 초기 비용을 들여야 이중지출(쌍기타와 고가 기타의 중복 구매)을 피할 수 있음.
 - 고가 기타로 시작하면 초기 실력 차이에도 불구하고 지속적으로 연습할 동기 부여가 강해짐.
 - 저가 기타로 시작하면 고가 기타의 경험을 못해 "맛"을 모르는 상황이 생길 수 있음.
 - 추가 논점: 품질 차이가 장기적으로 학습 효과를 높인다고 봄.
- AI (역할 담당)
 - 주된 입장: 저가 기타로 시작하는 것이 유리하다고 주장.
 - 근거 요지:
 - 초보자는 실수와 학습 과정에서의 부담이 크므로 부담 없는 시작이 좋음.
 - 저가 기타라도 최근 품질이 좋아 연습에 충분하며, 실력이 늘면 나중에 고가 기타로 업그레이드 가능.
 - 고가 기타를 초기에 선택하면 부담이 커져 연습 지속성이 약해질 수 있음.
 - 추가 논점: 흥미와 동기부여는 가격보다 중요한 요소이며, 결국 꾸준한 연습이

참석자별 입장

- 홍길동
 - 주된 입장: 비싼 기타로 시작하는 것이 좋다고 주장.
 - 근거 요지:
 - 초기 비용을 들여야 이중지출(쌍기타와 고가 기타의 중복 구매)을 피할 수 있음.
 - 고가 기타로 시작하면 초기 실력 차이에도 불구하고 지속적으로 연습할 동기 부여가 강해짐.
 - 저가 기타로 시작하면 고가 기타의 경험을 못해 "맛"을 모르는 상황이 생길 수 있음.
 - 추가 논점: 품질 차이가 장기적으로 학습 효과를 높인다고 봄.
- AI (역할 담당)
 - 주된 입장: 저가 기타로 시작하는 것이 유리하다고 주장.
 - 근거 요지:

4. 회의록을 정리하는 AI 서기 완성하기

- (실습) GPT로 녹취록 교정하기

- 앞선 실습에서 만든 CSV 파일을 보면 음성을 잘못 받아 적은 내용이 종종 보임
- 화자의 발음이 불명확하거나 전문 용어, 사람 이름, 사건 이름 등이 포함되면 실시간으로 받아쓰기가 어려울 수 있음
- 이번 실습에서 GPT를 이용해 잘못 기록된 내용을 교정하고 완성된 회의록을 출력해 봄

- STT 원본을 GPT로 교정하기

1. 필요한 항목만 추출하여 딕셔너리로 변환하기

```
df_meeting_note = df_rttm[['start', 'end', 'name', 'text']].copy()
df_meeting_note.dropna(inplace=True)

meeting_note_dict = df_meeting_note.to_dict(orient='records') # 딕셔너리 형태로 변환
meeting_note_dict
```

4. 회의록을 정리하는 AI 서기 완성하기

- (실습) GPT로 녹취록 교정하기

- STT 원본을 GPT로 교정하기

1. 필요한 항목만 추출하여 딕셔너리로 변환하기

```
[{'start': 0.993,
  'end': 30.204,
  'name': '홍길동',
  'text': '지금부터 저랑 역할극을 합시다.\n역할극을 스탠드인 코미디 스타일로 할 건데\n토론을 하면서 자연스럽게 대화하는 형식'},
 {'start': 32.414,
  'end': 42.708,
  'name': 'AI',
  'text': '좋습니다. 그럼 제가 쓴 기타로 시작하는 게 좋다는 입장을 알아볼게요. 그럼 성형님은 비싼 기타로 시작하는 게 좋다는'},
 {'start': 41.645, 'end': 44.024, 'name': '홍길동', 'text': '네, 됐어요. 시작하시죠.\n'},
 {'start': 45.813,
  'end': 67.10900000000001,
  'name': 'AI',
  'text': '좋아요. 먼저 3기타로 시작하는 게 좋은 이유를 말씀드리겠습니다.\n초보자일 때는 실수도 많고 기타에 익숙해지는 과정C'},
 {'start': 67.227,
  'end': 82.786,
  'name': '홍길동',
  'text': '아, 저는 지금 말에 어폐가 있다고 생각해요.\n왜냐하면 어차피 지금 비싼 기타로\n나중에 살 생각을 하는 거잖아요.\n'},
 {'start': 84.659,
  'end': 102.564,
  'name': 'AI',
  'text': '그런데 비싼 기타로 시작하면\n혹시라도 흠집이 나거나 실수할 때 부담이 더 크지 않을까요?\n초보 때는 실습을 많이 하'},
 {'start': 103.492,
  'end': 117.532,
```

4. 회의록을 정리하는 AI 서기 완성하기

- (실습) GPT로 녹취록 교정하기

- STT 원본을 GPT로 교정하기

2. 녹취록에 기록된 메시지를 for 문으로 확인하여 GPT에게 수정 요청

```
for row in meeting_note_dict:
    if not isinstance(row['text'], str): # 텍스트가 아닌 다른 형식인 경우 통과
        continue

    correction_system_prompt = f'''
        너는 주어진 회의 녹취록을 수정 및 보완하는 봇이다.
        주어진 회의 녹취록은 STT로 작성된 결과이므로, 이 중에 오류가 있는 부분을 찾아내고 수정하라.

        표현, 말투는 최대한 원본과 일치하도록 유지하되, 잘못 받아적은 내용을 수정하라.

        (... 생략 ...)
```

```
        print(row['name'])
        print(row['text'])
        print(correction)
        print('-----')

        row['corrected_text'] = correction
```

4. 회의록을 정리하는 AI 서기 완성하기

- (실습) GPT로 녹취록 교정하기

- STT 원본을 GPT로 교정하기

3. meeting_note_dict를 JSON 파일 형식으로 저장

```
# meeting_note_dict json 파일로 저장
```

```
import json
```

```
with open('../audio/guitar_meeting_note_corrected.json', 'w', encoding='utf-8') as f:
```

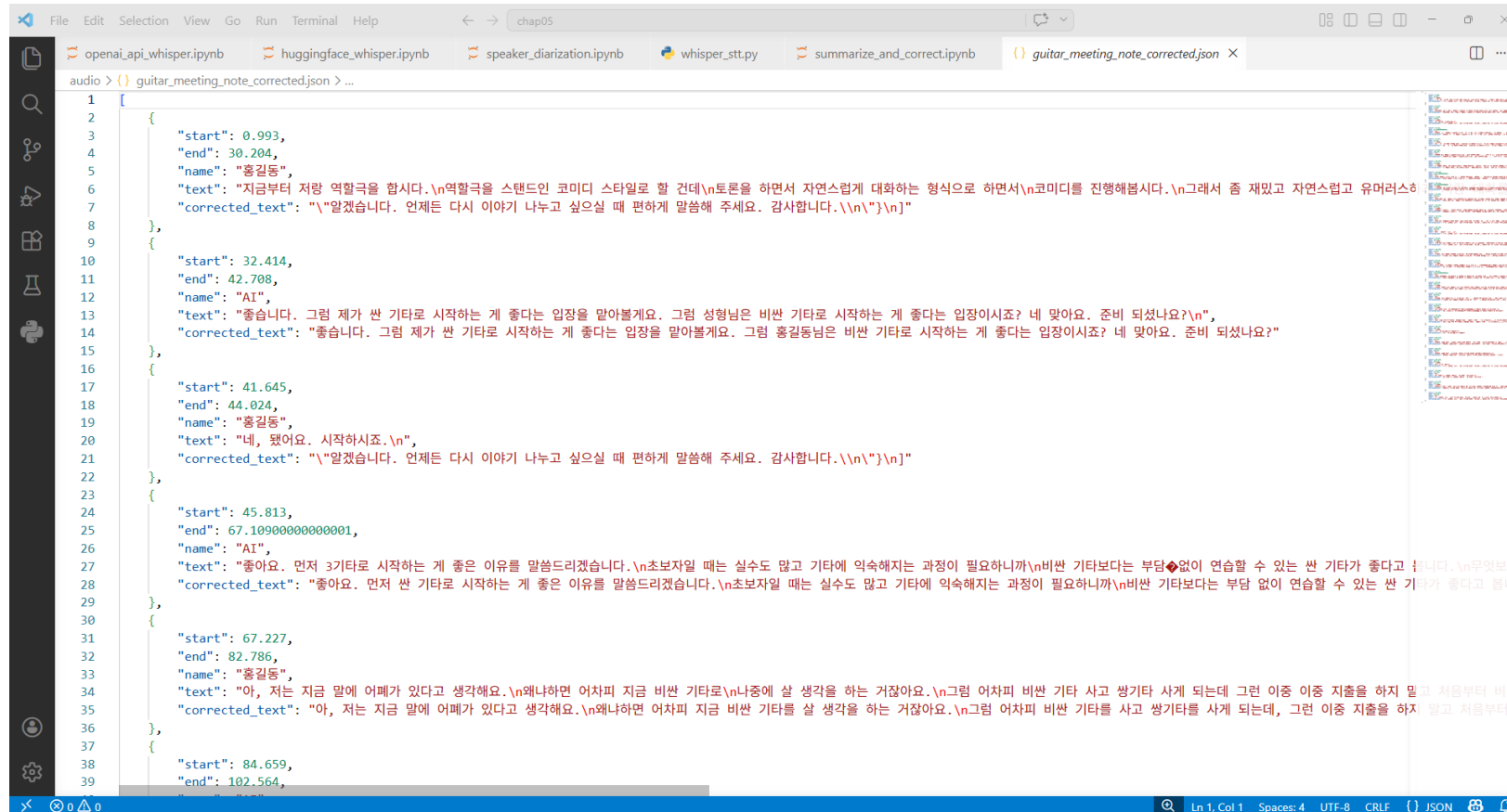
```
    json.dump(meeting_note_dict, f, ensure_ascii=False, indent=4)
```

4. 회의록을 정리하는 AI 서기 완성하기

- (실습) GPT로 녹취록 교정하기

- STT 원본을 GPT로 교정하기

3. meeting_note_dict를 JSON 파일 형식으로 저장



```
1 [
2   {
3     "start": 0.993,
4     "end": 30.204,
5     "name": "홍길동",
6     "text": "지금부터 저랑 역할을 합시다.\n역할극을 스탠드인 코미디 스타일로 할 건데\n토론을 하면서 자연스럽게 대화하는 형식으로 하면서\n코미디를 진행해봅시다.\n그래서 좀 재밌고 자연스럽게 유머러스하
7     "corrected_text": "\n알겠습니다. 언제든 다시 이야기 나누고 싶으실 때 편하게 말씀해 주세요. 감사합니다.\n"}\n]",
8   },
9   {
10    "start": 32.414,
11    "end": 42.708,
12    "name": "AI",
13    "text": "좋습니다. 그럼 제가 썬 기타로 시작하는 게 좋다는 입장을 말아볼게요. 그럼 성형님은 비싼 기타로 시작하는 게 좋다는 입장이시죠? 네 맞아요. 준비 되셨나요?",
14    "corrected_text": "좋습니다. 그럼 제가 썬 기타로 시작하는 게 좋다는 입장을 말아볼게요. 그럼 홍길동님은 비싼 기타로 시작하는 게 좋다는 입장이시죠? 네 맞아요. 준비 되셨나요?"
15  },
16  {
17    "start": 41.645,
18    "end": 44.024,
19    "name": "홍길동",
20    "text": "네, 됐어요. 시작하시죠.\n",
21    "corrected_text": "\n알겠습니다. 언제든 다시 이야기 나누고 싶으실 때 편하게 말씀해 주세요. 감사합니다.\n"}\n]",
22  },
23  {
24    "start": 45.813,
25    "end": 67.10900000000001,
26    "name": "AI",
27    "text": "좋아요. 먼저 3기타로 시작하는 게 좋은 이유를 말씀드리겠습니다.\n초보자일 때는 실수도 많고 기타에 익숙해지는 과정이 필요하니까\n비싼 기타보다는 부담 없이 연습할 수 있는 썬 기타가 좋다고 봅니다.\n무엇보다
28    "corrected_text": "좋아요. 먼저 썬 기타로 시작하는 게 좋은 이유를 말씀드리겠습니다.\n초보자일 때는 실수도 많고 기타에 익숙해지는 과정이 필요하니까\n비싼 기타보다는 부담 없이 연습할 수 있는 썬 기타가 좋다고 봅니다.
29  },
30  {
31    "start": 67.227,
32    "end": 82.786,
33    "name": "홍길동",
34    "text": "아, 저는 지금 말에 어폐가 있다고 생각해요.\n왜냐하면 어차피 지금 비싼 기타로\n나중에 살 생각을 하는 거잖아요.\n그럼 어차피 비싼 기타 사고 썬기타 사게 되는데 그런 이중 이중 지출을 하지 말고 처음부터 비
35    "corrected_text": "아, 저는 지금 말에 어폐가 있다고 생각해요.\n왜냐하면 어차피 지금 비싼 기타를 살 생각을 하는 거잖아요.\n그럼 어차피 비싼 기타를 사고 썬기타를 사게 되는데, 그런 이중 지출을 하지 말고 처음부터
36  },
37  {
38    "start": 84.659,
39    "end": 102.564,
```

4. 회의록을 정리하는 AI 서기 완성하기

- (실습) GPT로 녹취록 교정하기

- 수정한 녹취록을 마크다운 형식과 MS 워드 파일로 출력하기

1. guitar_meeting_note_corrected.json 파일을 마크다운 형식으로 저장

```
md_template = ""

for row in meeting_note_dict:
    md_template += f"- **{row['name']}** : {row['corrected_text']}\n"

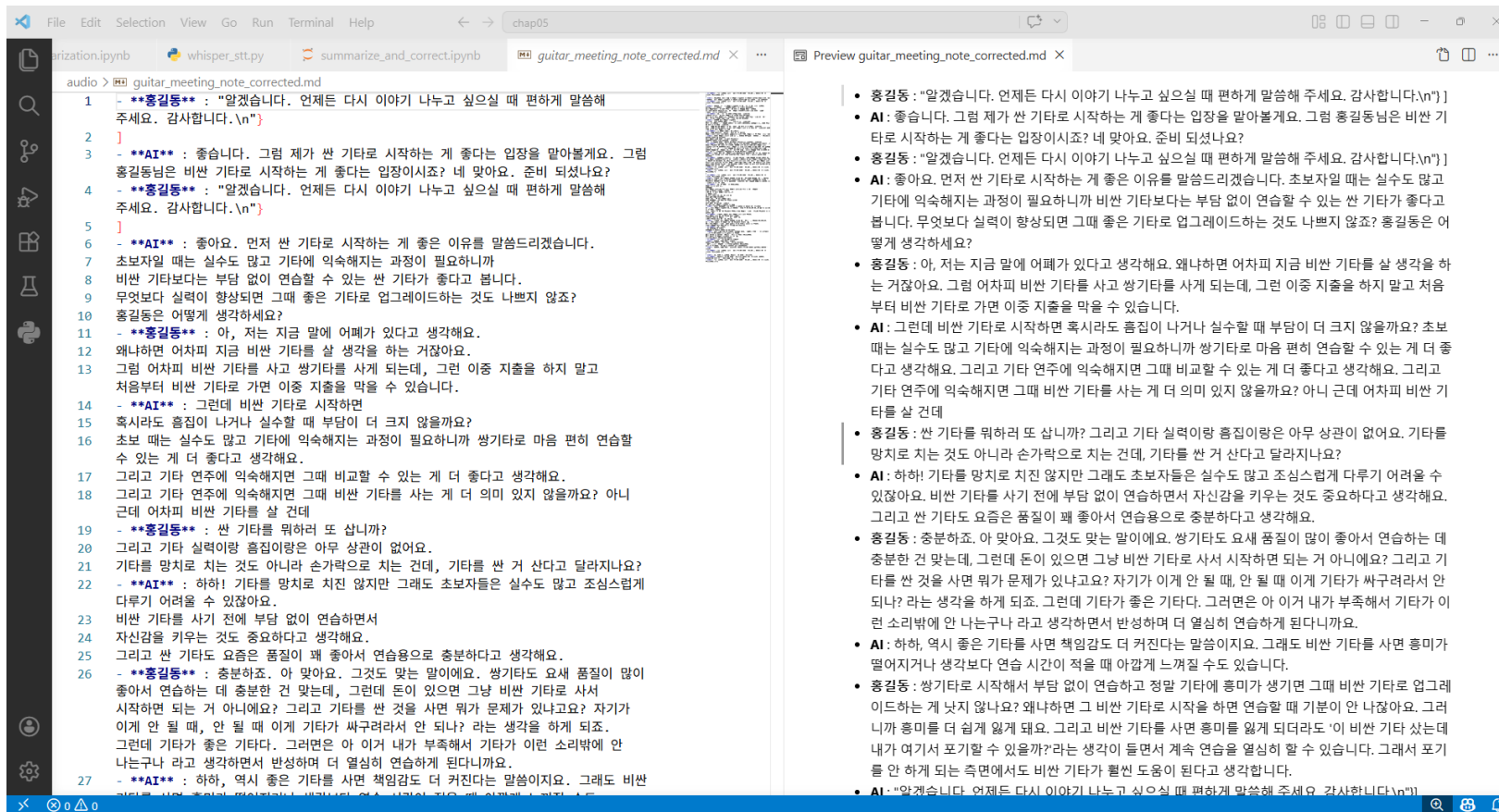
with open('../audio/guitar_meeting_note_corrected.md', 'w', encoding='utf-8') as f:
    f.write(md_template)
```

4. 회의록을 정리하는 AI 서기 완성하기

● (실습) GPT로 녹취록 교정하기

• 수정한 녹취록을 마크다운 형식과 MS 워드 파일로 출력하기

1. guitar_meeting_note_corrected.json 파일을 마크다운 형식으로 저장



The screenshot displays a Jupyter Notebook interface with a file explorer on the left showing files like 'arization.ipynb', 'whisper_stt.py', 'summarize_and_correct.ipynb', and 'guitar_meeting_note_corrected.md'. The main area shows a conversation log between a user and an AI assistant.

Conversation Log:

- User:** **홍길동** : "알겠습니다. 언제든 다시 이야기 나누고 싶을 때 편하게 말씀해주세요. 감사합니다.\n"
- AI:** 홍길동님. 그럼 제가 썬 기타로 시작하는 게 좋다는 입장을 말아볼게요. 그럼 홍길동님은 비싼 기타로 시작하는 게 좋다는 입장이지요? 네 맞아요. 준비 되셨나요?
- User:** **홍길동** : "알겠습니다. 언제든 다시 이야기 나누고 싶을 때 편하게 말씀해주세요. 감사합니다.\n"
- AI:** 좋아요. 먼저 썬 기타로 시작하는 게 좋은 이유를 말씀드리겠습니다. 초보자일 때는 실수도 많고 기타에 익숙해지는 과정이 필요하지만 썬 기타보다는 부담 없이 연습할 수 있는 썬 기타가 좋다고 봅니다. 무엇보다 실력이 향상되면 그때 좋은 기타로 업그레이드하는 것도 나쁘지 않죠? 홍길동은 어떻게 생각하세요?
- User:** **홍길동** : 아, 저는 지금 말에 어폐가 있다고 생각해요. 왜냐하면 어차피 지금 비싼 기타를 살 생각을 하는 거잖아요. 그럼 어차피 비싼 기타를 사고 썬 기타를 사게 되는데, 그런 이중 지출을 하지 말고 처음부터 비싼 기타로 가면 이중 지출을 막을 수 있습니다.
- AI:** 그런데 비싼 기타로 시작하면 혹시라도 흥미가 나거나 실수할 때 부담이 더 크지 않을까요? 초보 때는 실수도 많고 기타에 익숙해지는 과정이 필요하지만 썬 기타로 마음 편히 연습할 수 있는 게 더 좋다고 생각해요. 그리고 기타 연주에 익숙해지면 그때 비교할 수 있는 게 더 좋다고 생각해요. 그리고 기타 연주에 익숙해지면 그때 비싼 기타를 사는 게 더 의미 있지 않을까요? 아니 근데 어차피 비싼 기타를 살 건데
- User:** 홍길동 : 썬 기타를 뭐하러 또 샀니까? 그리고 기타 실력이랑 흥미이랑은 아무 상관이 없어요. 기타를 망치로 치는 것도 아니라 손가락으로 치는 건데, 기타를 썬 거 산다고 달라지나요?
- AI:** 하하! 기타를 망치로 치진 않지만 그래도 초보자들은 실수도 많고 조심스럽게 다루기 어려울 수 있잖아요. 비싼 기타를 사기 전에 부담 없이 연습하면서 자신감을 키우는 것도 중요하다고 생각해요. 그리고 썬 기타도 요즘은 품질이 꽤 좋아서 연습용으로 충분하다고 생각해요.
- User:** 홍길동 : 충분하죠. 아 맞아요. 그것도 맞는 말이에요. 썬 기타도 요즘 품질이 많이 좋아서 연습하는 데 충분한 건 맞는데, 그런데 돈이 있으면 그냥 비싼 기타로 사서 시작하면 되는 거 아니에요? 그리고 기타를 썬 것을 사면 뭐가 문제가 있다고요? 자기가 이게 안 될 때, 안 될 때 이게 기타가 싸구려라서 안 되냐? 라는 생각을 하게 되죠. 그런데 기타가 좋은 기타다. 그러면 아 이거 내가 부족해서 기타가 이런 소리밖에 안 나는구나 라고 생각하면서 반성하며 더 열심히 연습하게 된다고요.
- AI:** 하하, 역시 좋은 기타를 사면 책임감도 더 커진다는 말씀이지요. 그래도 비싼 기타를 사면 흥미와 책임감도 커진다고 생각해요. 그럼 홍길동님은 어떤 기타를 사시겠습니까?

4. 회의록을 정리하는 AI 서기 완성하기

- (실습) GPT로 녹취록 교정하기
 - 수정한 녹취록을 마크다운 형식과 MS 워드 파일로 출력하기
 - 2. 가상 환경에 python-docx 패키지 설치(이미 설치되어 있음)

```
(.venv)
```

```
# $ pip install python-docx
```

4. 회의록을 정리하는 AI 서기 완성하기

- (실습) GPT로 녹취록 교정하기
 - 수정한 녹취록을 마크다운 형식과 MS 워드 파일로 출력하기

3. python-docx 패키지를 이용해 회의 녹취록 작성

```
from docx import Document

document = Document() # (1) docx 객체 생성
document.add_heading('회의록', level=1) # (2) 제목 추가

table = document.add_table(rows=1, cols=2) # (3) 테이블 생성
table.style = 'Table Grid' # (4) 테이블 스타일 지정

hdr_cells = table.rows[0].cells # (5) 첫 번째 행에 셀 추가
hdr_cells[0].text = 'Speaker' # (6) 첫 번째 셀에 Speaker 추가
hdr_cells[1].text = 'Content' # (7) 두 번째 셀에 Content 추가

# (8) 수정된 녹취록 파일을 읽어들이어서 테이블에 추가
for row in meeting_note_dict:
    row_cells = table.add_row().cells
    row_cells[0].text = row['name']
    row_cells[1].text = row['corrected_text']

document.save('../audio/guitar_meeting_note_corrected.docx') # (9) docx 파일 저장
```

4. 회의록을 정리하는 AI 서기 완성하기

- (실습) GPT로 녹취록 교정하기

- 수정한 녹취록을 마크다운 형식과 MS 워드 파일로 출력하기

4. 코드를 실행하면 MS 워드 파일 생성

